Assignment: Building a QA Chatbot on Ancient Greece Using Advanced AI Frameworks

Role: AI Engineer at FAB (First Abu Dhabi Bank)
Objective: Develop a robust and accurate QA (Question-Answering) chatbot capable of answering questions about Ancient Greece using a provided dataset. The chatbot must minimize hallucinations (fabricated facts) and ensure all answers are directly supported by the data. Bonus points will be awarded for creativity, source citation, and the implementation of robust evaluation metrics.

Deadline: 7/Aug/2025 Thursday, 6:00 PM UAE Time

Task Description

As a Data Scientist at FAB, your task is to create a QA chatbot that can answer questions about Ancient Greece using a dataset provided in the ancient_greece_data folder. This dataset consists of text files corresponding to pages of an AI-generated textbook. The chatbot must ensure that all answers are directly supported by the data. If the data does not contain sufficient information to answer a question, the chatbot should explicitly state this.

You are encouraged to use the **LlamaIndex**, **LangChain**, and **Pinecone** frameworks, **but you may use any tools you are comfortable** with. Creativity in prompt engineering, RAG (Retrieval-Augmented Generation), and LLM (Large Language Model) chaining is highly encouraged.

Requirements

1. **Data Preparation:**
   - Load and preprocess the text files from the ancient_greece_data folder.
   - Use a vector database (e.g., Pinecone) to store embeddings for efficient retrieval.
   - Ensure the preprocessing pipeline handles noise, special characters, and tokenization effectively.
2. **Chatbot Development:**
   - Implement a QA system using **LlamaIndex** and **LangChain**.
   - Ensure the chatbot retrieves relevant information from the dataset and generates answers backed by the data.
   - If the data does not contain enough information to answer a question, the chatbot should respond with: *"I don't have enough information to answer this question."*
3. **Source Citation (Bonus):**

- Modify the chatbot to cite the specific text file(s) or sections it used to generate the answer. For example: *"This answer is based on information from File_12.txt."*
- Ensure the citation mechanism is accurate and user-friendly.

4. **Evaluation Metrics (Bonus):**
   - Assess the chatbot's performance using standard metrics or creative metrics (e.g., user satisfaction, hallucination rate).
   - Provide a detailed analysis of the chatbot's strengths and weaknesses, including potential areas for improvement.
5. **LLM and Embedding Options:**
   - Use an Closed/Open source LLMs (e.g., LLaMA, GPT-J).
   - Use open-source embeddings (e.g., **Sentence Transformers**) for cost efficiency and flexibility.
   - Justify your choice of LLM and embedding model in your final report.
6. **Deliverables:**
   - A **Jupyter Notebook** with detailed steps, code, and comments explaining your thought process.
   - A **presentation** or **report** summarizing your approach, challenges, and results.
   - If applicable, provide a link to a deployed version of the chatbot.

Bonus Points

1. **Source Citation:**
   - Implement a feature where the chatbot cites the specific text file(s) or sections used to generate the answer.
2. **Creative Metrics:**
   - Propose and implement unique evaluation metrics to assess the chatbot's performance.
3. **Deployment:**
   - Deploy the chatbot as a web application or API for interactive use.

Example Workflow

1. **Data Loading:**
   - Load the text files from the ancient_greece_data folder.
   - Preprocess the text (e.g., remove special characters, tokenize).
2. **Embedding Generation:**
   - Generate embeddings for the text using a pre-trained model (e.g., **Sentence Transformers**).
   - Store the embeddings in a vector database (e.g., Pinecone).
3. **QA Pipeline:**

   - Use framework to create an index of the embeddings.
   - Implement pipeline to retrieve relevant documents and generate answers.
4. **Prompt Engineering:**
   - Experiment with different prompts to optimize the chatbot's performance.
   - Test the chatbot with a set of sample questions and evaluate its accuracy and reliability.
5. **Evaluation:**
   - Test the chatbot with a set of sample questions and evaluate its accuracy and reliability.
   - Use both quantitative and qualitative metrics to assess performance.

Submission Guidelines

1. Submit a **Jupyter Notebook** with all code, comments, and explanations.
2. Include a **report** or **presentation** summarizing your approach, results, and insights.
3. If applicable, provide a link to a deployed version of the chatbot.

Grading Criteria

1. **Functionality (40%):**
   - Does the chatbot answer questions accurately and reliably?
   - Does it handle cases where the data is insufficient?
2. **Creativity (20%):**
   - How creative is the implementation (e.g., prompt engineering, RAG pipeline)?
   - Are there any unique features (e.g., source citation, hallucination detection)?
3. **Evaluation (20%):**
   - Are the evaluation metrics well-defined and meaningful?
   - Is there a thorough analysis of the chatbot's performance?
4. **Presentation (20%):**
   - Is the Jupyter Notebook well-documented and easy to follow?
   - Is the report/presentation clear and concise?

Tools and Resources

- **Frameworks:** LlamaIndex, LangChain, Pinecone or other.
- **LLMs:** OpenAI API, LLaMA, GPT-J, Claude 3.5 Haiku or other.
- **Embedding Models:** Sentence Transformers or other.
- **Evaluation Libraries:** ROUGE, BLEU, or custom metrics.

This assignment is designed to challenge your skills as a data scientist and encourage creativity in solving real-world problems. Good luck!